SST: Multi-Scale Hybrid Mamba-Transformer Experts for Time Series Forecasting

Xiongxiao Xu
Illinois Institute of Technology
Chicago, IL, USA
xxu85@hawk.illinoistech.edu

Canyu Chen
Illinois Institute of Technology
Chicago, IL, USA
cchen151@hawk.illinoistech.edu

Yueqing Liang
Illinois Institute of Technology
Chicago, IL, USA
yliang40@hawk.illinoistech.edu

Baixiang Huang
Emory University
Atlanta, GA, USA
baixiang.huang@emory.edu

Guangji Bai
Emory University
Atlanta, GA, USA
gbai5@emory.edu

Liang Zhao
Emory University
Atlanta, GA, USA
liang.zhao@emory.edu

Kai Shu
Emory University
Atlanta, GA, USA
kai.shu@emory.edu

Abstract

Time series forecasting has made significant advances, including with Transformer-based models. The attention mechanism in Transformer effectively captures temporal dependencies by attending to all past inputs simultaneously. However, its quadratic computational complexity with respect to sequence length limits the scalability for long-range modeling. Recent state space models (SSMs) such as Mamba offer a promising alternative by achieving linear complexity without attention. Yet, Mamba compresses historical information into a fixed-size latent state, potentially causing information loss and limiting representational effectiveness. This raises a key research question: *Can we design a hybrid Mamba-Transformer architecture that is both effective and efficient for time series forecasting?* To address it, we adapt a hybrid Mamba-Transformer architecture *Mambaformer*, originally proposed for language modeling, to the time series domain. Preliminary experiments reveal that naively stacking Mamba and Transformer layers in Mambaformer is suboptimal for time series forecasting, due to an *information interference* problem. To mitigate this issue, we introduce a new time series decomposition strategy that separates time series into long-range patterns and short-range variations. Then we show that Mamba excels at capturing long-term structures, while Transformer is more effective at modeling short-term dynamics. Building on this insight, we propose *State Space Transformer (SST)*, a multi-scale hybrid model with expert modules: a Mamba expert for long-range patterns and a Transformer expert for short-term variations. To facilitate learning the patterns and variations, SST employs a multi-scale patching mechanism to adaptively adjust time series resolution: low resolution for long-term patterns and high resolution for short-term variations. Comprehensive experiments on real-world datasets demonstrate that SST achieves state-of-the-art performance while scaling linearly with sequence length ($O(L)$). The code is available here[1].

CCS Concepts

• **Computing methodologies** → **Machine learning approaches**.

Keywords

Time Series Forecasting, Mamba, Transformer

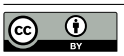
ACM Reference Format:
Xiongxiao Xu, Canyu Chen, Yueqing Liang, Baixiang Huang, Guangji Bai, Liang Zhao, and Kai Shu. 2025. SST: Multi-Scale Hybrid Mamba-Transformer Experts for Time Series Forecasting. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25), November 10–14, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3746252.3761394

1 Introduction

Time series forecasting has long served as a foundational task in a wide range of real-world applications [50], including weather forecasting [59], stock prediction [5], and scientific computing [49]. For example, in high-performance computing (HPC), ML-based surrogate models are trained to accelerate large-scale simulations of supercomputers by predicting their billion or even trillion behaviors over various timescales [8], thereby avoiding a huge amount of energy consumption of supercomputers [48].

Transformer-based and large language model (LLM)-based forecasters [19, 30], which are pre-trained on Transformer architectures, have demonstrated superiority in time series forecasting. The core strength of Transformer models lies in their attention mechanism, which identifies the most relevant information across the entire input sequence, and captures complex time series dependencies. However, the quadratic computation complexity of the attention limits its scalability for long-range sequence modeling. Recent advancements in state space models (SSMs), such as Mamba [13],

[1] https://github.com/XiongxiaoXu/SST



CIKM '25, Seoul, Republic of Korea

ACM ISBN 979-8-4007-2040-6/2025/11
https://doi.org/10.1145/3746252.3761394

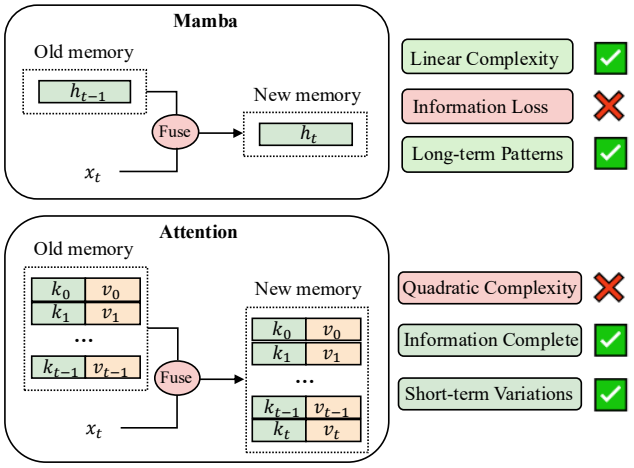


Figure 1: Comparison of memory mechanism between Mamba and attention, where x_i denotes the input token of $i-th$ step. Top: Mamba, as a RNN-like mechanism, compresses previous tokens into a fixed-size state h_{t-1}, which serves as the memory. When the current token x_t occurs, x_t is incorporated into h_{t-1}, leading to a new memory h_t with the same size. This fixed size means that the memory is inherently lossy but linearly efficient. Bottom: Attention stores all previous tokens' keys k and values v as memory. The memory is updated by continuously adding the current token's key and value, so the memory is lossless. Therefore, attention can effectively manage short sequences but may encounter computational difficulties with longer ones.

attempt to address this limitation by introducing linear-complexity through integrating gating, convolutions, and input-dependent token selection. While Mamba reduces the computational cost of the attention, it memorizes information by compressing the past information into a fixed-size latent state, which can result in information loss and constrained performance. Moreover, recent findings show that SSMs and Transformers are complementary for language modeling [10, 24, 31]. Motivated by it, we pose a research question: *Can we design a hybrid Mamba-Transformer architecture that is both effective and efficient for time series forecasting?*

However, it presents several significant challenges for hybrid Mamba-Transformer model in time series forecasting. First, while Mamba and Transformer each have distinct strengths, i.e., Mamba offers efficiency, and Transformer excels at expressivity, their limitations are also complementary. It remains unclear how to design a hybrid model that leverages their respective advantages without inheriting their drawbacks. Second, both models originate from language modeling, and how to embed time series–specific inductive biases into a hybrid architecture remains an open question. A straightforward approach is to adapt *Mambaformer* [31], which is a hybrid Mamba-Transformer framework initially developed in language modeling. Mambaformer achieves strong performance in text modeling by stacking Mamba layers with attention blocks. Given the sequential nature shared by language and time series data, one might expect Mambaformer to generalize reasonably well to time series forecasting. However, our preliminary results suggest

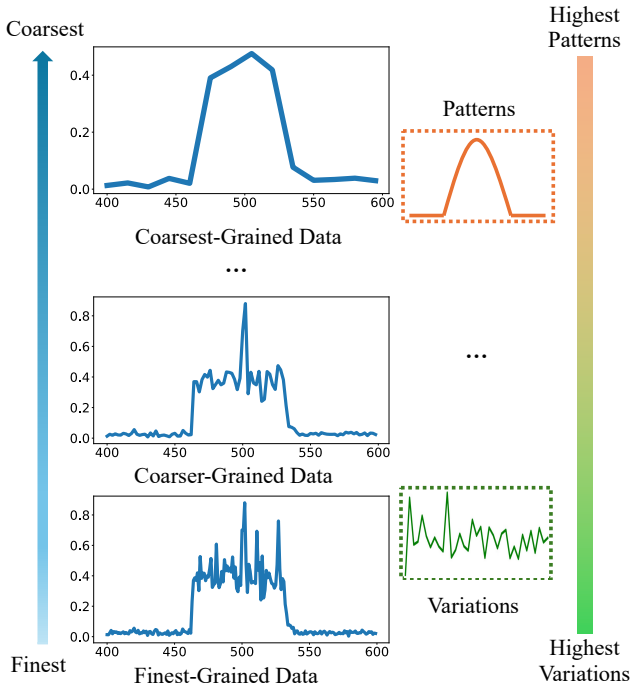


Figure 2: From bottom to top, as time series resolution shifts from fine-grained to coarse-grained, patterns become increasingly pronounced while variations diminish.

that the naively interleaving of Mamba and Transformer layers in Mambaformer even fails to outperform a simple linear baseline *DLinear*[54]. We attribute this degradation to an *information interference* issue where Mamba and attention mechanisms conflict in how they encode and retrieve temporal information.

To address this issue, we introduce a new time series decomposition method that breaks a sequence into long-range patterns and short-range variations, which can be modeled independently by Mamba and Transformer, respectively. This strategy decouples the Mamba-Transformer stack, thus avoiding the *information interference* issue. Mamba, which maintains knowledge by a fixed-size latent state inherited from recurrent neural networks (RNNs) [29] but may lose information, is well-suited for capturing long-term patterns with filtering out variations in the long run. In contrast, Transformer, which does not compress context into a smaller state but is prohibitively computing expensive, is more effective in capturing short-term variations. To harness their complementary strengths, we propose a multi-scale hybrid Mamba-Transformer experts model *State Space Transformer (SST)*.

The core idea of SST is to separately model patterns in long range by Mamba and variations in short range by Transformer. Specifically, SST includes three components: (a) multi-scale patcher, (b) hybrid Mamba-Transformer experts, and (c) long-short router. (a) The multi-scale patcher modifies resolutions of patched time series (PTS) [30]: low-resolution for long range, and high-resolution for short range. Figure 2 illustrates that patterns are more discernible at a coarser granularity, while variations emerge at a finer granularity. Therefore, we employ larger patches and longer stride for long range to obtain a low-resolution PTS; smaller patches and shorter

stride on short term leads to a high-resolution PTS. Moreover, since existing literature lacks a quantitative metric to assess the resolution of PTS, we propose a metric R_{PTS} to precisely quantify the granularity of PTS. (b) Inspired by the idea of Mixture-of-Experts (MoEs) [11], SST adopts Mamba as an expert to model patterns in long range and Transformer as the other expert to capture variations in short range. Unlike traditional MoEs' approaches where roles of experts are ambiguous, we clearly delineates the responsibilities for two experts. (c) To adaptively integrate the two specialized experts, the long-short router is proposed to learn their contributions, thus leading to enhanced forecasting performance. Remarkably, SST achieves SOTA performance and also ensures a linear complexity $O(L)$ on input length L. To the best of our knowledge, SST is the first hybrid Mamba-Transformer architecture in time series forecasting. We summarize our contributions as follows:

- We investigate the potential of hybrid Mamba-Transformer architectures for time series forecasting by adapting Mambaformer to this domain. Our analysis reveals that naively stacking Mamba and Transformer layers, as done in Mambaformer, leads to an information interference issue, resulting in suboptimal forecasting performance.
- We introduce a new time series decomposition method that separates input sequences into long-range patterns and short-range variations, enabling Mamba and Transformer modules to function independently, thus eliminating information interference issue inherent in naive hybrid architectures.
- We propose a novel hybrid Mamba-Transformer experts architecture SST that leverages Mamba as a long-term patterns expert and Transformer as a short-term variations expert. SST also incorporates a multi-scale patching mechanism to adjust time series resolution and a long-short routing module to adaptively fuse the expert outputs. Extensive experiments show that SST achieves state-of-the-art forecasting performance with linear complexity $O(L)$ in sequence length L.

2 Preliminaries

2.1 Problem Statement

In time series forecasting, historical time series with a look-back window $\mathcal{L} = (\mathbf{x}_1, \mathbf{x}_2, .., \mathbf{x}_L) \in \mathbb{R}^{L \times M}$ of length L is given, where each $\mathbf{x}_t \in \mathbb{R}^M$ at time step t is with M variates, we aim to forecast F future values $\mathcal{F} = (\mathbf{x}_{L+1}, \mathbf{x}_{L+2}, .., \mathbf{x}_{L+F}) \in \mathbb{R}^{F \times M}$ with forecasting length F. Long-range time series $\mathcal{L}$ denotes the full range of look-back window $\mathcal{L}[:]$, and short-range time series $\mathcal{S} \in \mathbb{R}^{S \times M}$ denotes the partial latest range $\mathcal{L}[-S:]$, $S < L$. Input $\mathcal{L}$ and output $\mathcal{F}$ can also be denoted as $(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, .., \mathbf{x}^{(M)})$ indicating M dimensions.

2.2 State Space Models

The State Space Model (SSM) is a class of sequence modeling frameworks that are broadly related to RNNs, and CNNs, and classical state space models [16]. They are inspired by a continuous system that maps an input function $x(t) \in \mathbb{R}$ to an output function $y(t) \in \mathbb{R}$ through an implicit latent state $h(t) \in \mathbb{R}^N$ as follows:

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), y(t) = \mathbf{C}h(t) \tag{1}$$

where $\mathbf{A} \in \mathbb{R}^{N \times N}$, $\mathbf{B} \in \mathbb{R}^{N \times 1}$, and $\mathbf{C} \in \mathbb{R}^{1 \times N}$ are learnable matrices. SSM can be discretized from continuous signals into discrete sequences by a step size Δ as follows:

$$h_t = \overline{\mathbf{A}}h_{t-1} + \overline{\mathbf{B}}x_t, y = \mathbf{C}h_t \tag{2}$$

The discrete parameters $(\overline{\mathbf{A}}, \overline{\mathbf{B}})$ can be obtained from continuous parameters $(\Delta, \mathbf{A}, \mathbf{B})$ through a discretization rule, such as zero-order hold (ZOH) rule $\overline{\mathbf{A}} = \exp(\Delta\mathbf{A})$, $\overline{\mathbf{B}} = \exp(\Delta\mathbf{A})^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I}) \cdot \Delta\mathbf{B}$. After discretization, the model can be computed in two ways, either as a linear recurrence for inference as shown in Equation 2, or as a global convolution for training as the following Equation 3 shows, where $\overline{\mathbf{K}}$ is a convolution kernel:

$$\overline{\mathbf{K}} = (\mathbf{C}\overline{\mathbf{B}}, \mathbf{C}\overline{\mathbf{A}\mathbf{B}}, ..., \mathbf{C}\overline{\mathbf{A}}^k\overline{\mathbf{B}}, ...), y = x * \overline{\mathbf{K}} \tag{3}$$

S4 is a structured SSM where the specialized Hippo [14] structure is imposed on the matrix $\mathbf{A}$ to capture long-range dependency. Building upon S4, Mamba [13] incorporates a selective SSM to propagate or forget information along the sequence, and a hardware-aware algorithm for efficient implementation.

2.3 Transformer

Transformer [39] is becoming a basis architecture for deep learning. The core module of Transformer is the multi-head attention. Each head in the layer simultaneously transforms input into query matrices Q, key matrices K, and value matrices V. The output is based on the scaled dot-product:

$$Attention(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = softmax(\frac{QK^T}{\sqrt{d_k}})V \tag{4}$$

where $\sqrt{d_k}$ is a scaling factor to avoid small gradients. The attention mechanism has demonstrated effectiveness in a wide range of applications; however, the quadratic complexity poses a significant challenge for scaling to long input.

2.4 Time Series Decomposition

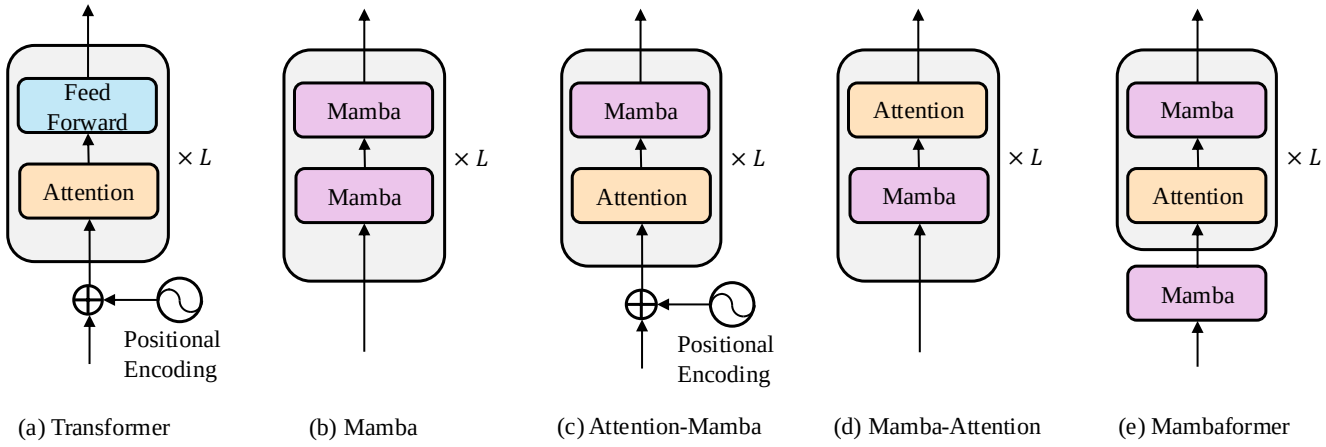
Time series decomposition is a classical technique that separates a time series into multiple interpretable components. One widely used method is STL (Seasonal-Trend decomposition using LOESS) [7], which decomposes a time series into three additive components: trend, seasonal, and residual (noise). This separation enables models to focus on distinct temporal dynamics of different scales. Formally, a univariate time series $\mathbf{x}$ (ignore superscript) can be expressed as:

$$\mathbf{x} = \mathbf{T} + \mathbf{S} + \mathbf{R} \tag{5}$$

where $\mathbf{T}$ captures the long-term trend, $\mathbf{S}$ represents seasonal or periodic patterns, and $\mathbf{R}$ denotes the residual fluctuations. STL decomposition is commonly used in time series literature [37, 57], where understanding and isolating periodicity or trends is critical for accurate forecasting and decision-making.

3 Is Vanilla Stacking of Mamba-Transformer Effective for Time Series Forecasting?

In this section, we evaluate the effectiveness of the hybrid *Mambaformer* architecture in time series, which is originally proposed for language modeling. Our experiments reveal that a straightforward stacking of Mamba and Transformer layers struggles to model temporal dependencies in time series data, potentially due to an



(a) Transformer (b) Mamba (c) Attention-Mamba (d) Mamba-Attention (e) Mambaformer

Figure 3: The architectures of Mambaformer family. Positional encoding is optional across these variants. Mamba layers inherently encode positional information by state dynamics while Transformer layers require explicit positional encoding. When a Mamba layer is before an attention layer (Mamba, Mamba-Attention, and Mambaformer), positional encoding can be omitted. However, if the attention layer comes first (Transformer and Attention-Mamba), positional encoding is necessary.

information interference issue. To address it, we propose to decompose time series into long-range patterns and short-range variations, which can be addressed separately by Mamba and Transformer.

3.1 Adapting Mambaformer to Time Series

We adapt the Mambaformer architecture [31] to time series forecasting. It attempts to combine strengths of Mamba and attention by interleaving their layers. To make Mambaformer compatible with time series data, we introduce two types of embedding layers:

- **Conv**: A one-dimensional convolutional encoder that projects raw time series into a high-dimensional space while preserving local semantics [4].
- **PI (Patch + Instance Normalization)**: We employ a time series encoding methods in PatchTST [30]. It utilizes a patching strategy to reduce sequence length and preserve local structure, coupled with instance normalization [20, 38] to reduce distribution shift between training and testing.

The embedding layer is followed by the core Mambaformer architecture (Figure 3) to learn complex temporal dependencies. Finally, a linear layer maps the high-dimensional embeddings back to the original time series dimension to obtain forecasting results. Figure 3 illustrates five architectural variants of Mambaformer, including the original Transformer and Mamba, as well as hybrid combinations (Attention-Mamba, Mamba-Attention, and Mambaformer). We call them Mambaformer family in this work.

3.2 Mambaformer is Ineffective for Time Series

We compare Mambaformer family against an embarrassingly simple linear model *DLinear* [54]. Table 1 presents the experimental results, where the look-back window is fixed at $L = 196$, and performance is averaged across multiple forecasting horizons $F \in \{96, 192, 336, 720\}$. The results indicate that all Mambaformer family variants perform notably worse than the simple baseline DLinear. For example, on the ETTh1 dataset, DLinear achieves an MSE of 0.455, whereas the best-performing Mambaformer variant Mambaformer (PI) reaches 0.693. It suggests that naively stacking Mamba and Transformer layers is ineffective in time series forecasting, without accounting for the uniqueness of time series data.

Methods	ETTh1		Weather		Traffic	
Metrics	MSE	MAE	MSE	MAE	MSE	MAE
DLinear	**0.455**	**0.451**	**0.265**	**0.316**	**0.624**	0.383
Transformer (Conv)	0.991	0.790	0.491	0.485	0.672	0.397
Transformer (PI)	0.652	0.731	0.314	0.467	0.648	0.385
Mamba (Conv)	1.417	0.914	0.890	0.677	1.097	0.558
Mamba (PI)	1.172	0.662	0.561	0.419	0.965	0.415
Attention-Mamba (Conv)	0.995	0.792	0.656	0.591	0.661	0.394
Attention-Mamba (PI)	0.735	0.659	0.482	0.463	0.629	**0.358**
Mamba-Attention (Conv)	0.973	0.727	0.798	0.671	0.935	0.488
Mamba-Attention (PI)	0.741	0.663	0.569	0.454	0.843	0.441
Mambaformer (Conv)	0.962	0.721	0.842	0.679	0.733	0.401
Mambaformer (PI)	0.693	0.645	0.527	0.432	0.681	0.374

Table 1: Performance comparison between DLinear and Mambaformer family. The best performance is highlighted in bold. It shows that Mambaformer family cannot outperform DLinear, suggesting that the vanilla stacking of Mamba and Transformer is not effective for time series forecasting.

We attribute the poor performance of Mambaformer family to *information interference* issue in time series, where Mamba and attention mechanisms conflict in how they encode and retrieve temporal information. Specifically, Mamba compresses historical inputs into a fixed-size hidden state, inevitably discarding some information. When Transformer's attention mechanism is applied to this lossy representation, its effectiveness is diminished, since it can no longer attend to fine-grained details. Such degradation may be tolerable in text, where semantically meaningful tokens provide redundancy. However, time series data lacks such inherent structure, making it more sensitive to information loss. Therefore, a more principled and task-specific integration of sequential state space modeling and attention mechanisms is needed to fully exploit their potential in time series forecasting.

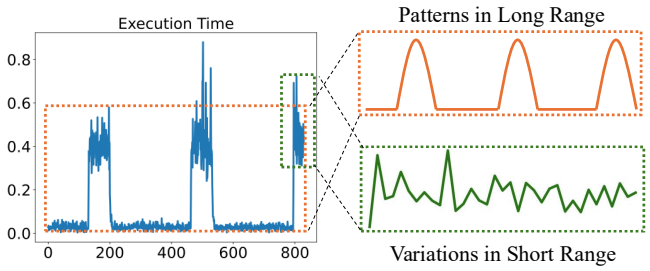


Figure 4: A real-world HPC dataset reflecting supercomputers' behaviors can be decomposed into patterns and variations by ranges. Long-term patterns (orange line) indicate repeated up-and-down trends because supercomputers intermittently executes applications, and short-term variations (green line) are extreme execution times caused by sudden network congestion.

3.3 Mamba for Long-Range Patterns and Transformer for Short-Range Variations

The issue of *information interference* arises from the naive stacking of Transformer and Mamba architectures. To address this, we propose a new time series strategy to separately model long-term patterns and short-term variations, which can be addressed by Mamba and Transformer independently, thus avoiding their conflicts.

We first describe the decomposition technique with a real-world HPC dataset [49] which describes execution times of supercomputers across different simulation iterations in the scientific computing domain. Figure 4 shows that time series can naturally be decomposed into long-range patterns and short-range variations. In this scenario, long-term patterns represent repeated up-and-down trends as supercomputers intermittently executes applications, and short-term variations are extreme execution times caused by sudden network congestion. For long-term forecasting, the model should focus on overall patterns while ignoring local deviations such as outliers. In contrast, short-term forecasting benefits from emphasizing local fluctuations, where patterns are less salient. For example, occasional spikes due to network congestion represent anomalies within the broader trend but are critical for predicting the immediate next-term system behavior [47].

Note that our decomposition is actually a variant of the classical STL decomposition. Unlike STL, which treats trend, seasonality, and residuals as symmetric components, our approach explicitly reflects the asymmetry between long- and short-range dependencies. Formally, we reinterpret STL as $\mathbf{x} = \mathbf{T}+\mathbf{S}+\mathbf{R} = (\mathbf{T}_{Long}+\mathbf{S}_{Long})+\mathbf{R}_{Long} \approx \mathbf{Patterns}_{Long} + \mathbf{Variations}_{Short}$, where long-range patterns encompass both trend and seasonality, and short-range variations represent noise or high-frequency signals that are contextually meaningful only in the short term.

As discussed in Figure 1, Mamba and Transformer differ fundamentally in memory mechanisms and temporal modeling. Mamba scales linearly with input length but inherently compresses information, making it a desirable property for capturing salient long-range patterns while ignoring noise. On the other hand, Transformer retains all historical inputs through key-value attention, enabling fine-grained modeling of short-term variations. Built on these complementary strengths, we propose a design principle for hybrid architectures: Mamba for long-range pattern extraction and Transformer for short-range variation modeling.

4 Methodology

In this section, we introduce the proposed multi-scale hybrid architecture *(State Space Transformer) SST*. As shown in Figure 5, SST primarily includes three modules: multi-scale patcher, hybrid Mamba-Transformer experts, and long-short router. Multi-scale patcher transforms input time series into distinct granularities according to ranges. Mamba extracts patterns from coarse-grained long-range series, and Transformer captures variations from fine-grained short-range series. The long-short router dynamically learns contributions of the two experts, finally leading to enhanced performance.

4.1 Multi-Scale Resolutions

Patching. Figure 2 suggests that time series patterns emerge when viewed at a broader-scale granularity, while variations become clearer when examined at a finer-scale granularity. Motivated by this, we propose a multi-scale patcher to modify resolutions of long- and short-range time series, i.e., low resolution for long range and high resolution for short range. The patching technique [30] is increasingly popular in time series analysis as it can retain local semantic information and reduce computation overhead by aggregating time steps into subseries-level patches. We modify time series resolution by adjusting the patch and stride length. Formally, input time series $\mathcal{L}$ is divided into M univariate time series $\mathcal{L} = (\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, .., \mathbf{x}^{(M)}) \in \mathbb{R}^{L\times M}, \mathbf{x}^{(i)} \in \mathbb{R}^{L\times 1}$. The patching process involves two factors: the patch length P and the stride length Str (the interval between the end point of two consecutive patches). Accordingly, the number of patches is $N = \lfloor \frac{L-P}{Str} \rfloor + 1$, and the patcher for each variate $\mathbf{x}^{(i)}$ generates a sequence of patches $\mathbf{x}_p^{(i)} \in \mathbb{R}^{N\times P}$, named patches time series (PTS). Note that for unpatched time series $\mathbf{x}^{(i)} \in \mathbb{R}^{L\times 1}$, L is the sequence dimension and 1 denotes the variate dimension. Correspondingly, for PTS $\mathbf{x}_p^{(i)} \in \mathbb{R}^{N\times P}$, N is the new sequence dimension and P is the new variate dimension.

Resolution. Intuitively, larger patches P, longer stride Str, and accordingly less number of patches N indicates a low resolution. We adopt such setting to generate a low-resolution time series for long range. It allows SST focus on modeling long-term patterns and ignoring small fluctuation. By contrast, smaller patches and shorter stride imply a high resolution. It enables SST to depict short-term nuances. Although the patching is becoming popular in time series community, no existing work tries to quantify resolutions of PTS. To mitigate the gap, we define a new metric as follows:

Definition 4.1. PTS Resolution. Let P, Str, and N denote patch length, stride length, and number of patches for a patched time series. The resolution of a PTS is defined as $R_{PTS} = \frac{\sqrt{P}}{Str}$.

The definition of PTS resolution takes two factors into account. First, PTS resolution aims to quantify the relative granularity of a PTS compared to its unpatched one. Second, PTS resolution describes the amount of temporal information of a PTS, in both the sequence dimension and variate dimension. Similar to the definition

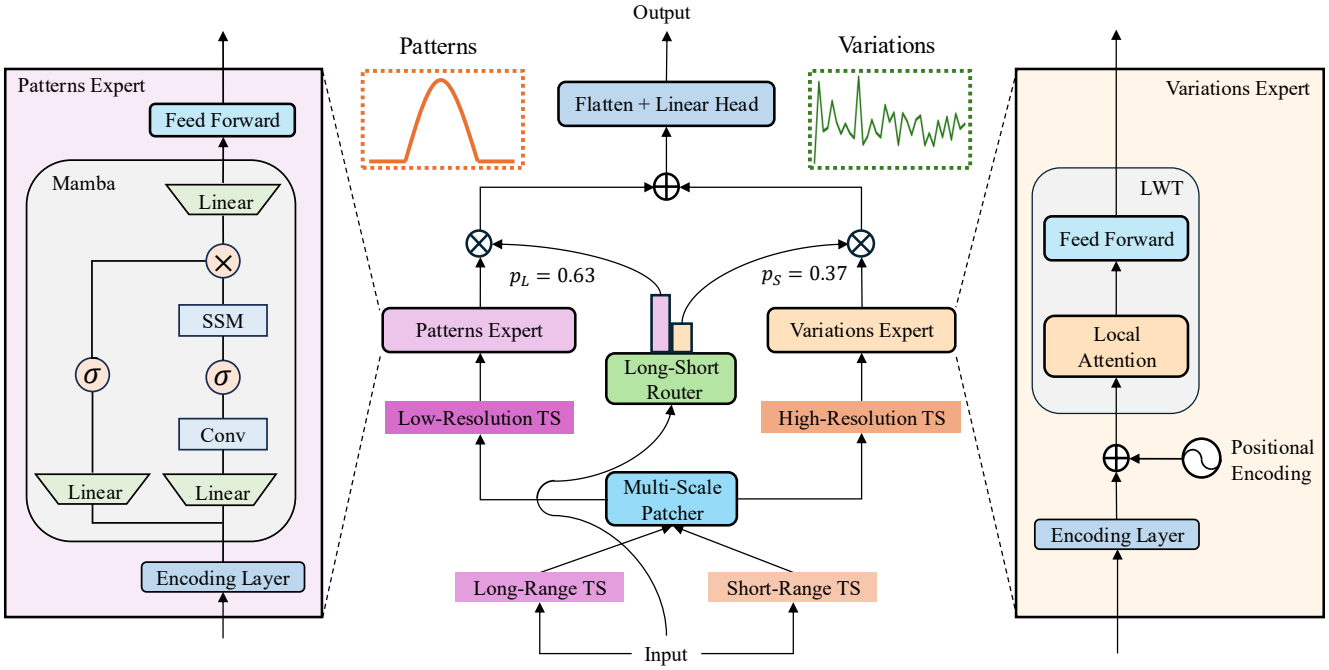


Figure 5: The overview of the SST. The multi-scale patcher transforms input time series in different resolutions according to ranges. The Mamba is dedicated for long-term patterns and the LWT is responsible for short-term variations. The long-short router adaptively learns the contributions of the two experts.

of the image resolution [3] equaling to $|width| \times |height|$, we multiply the two dimensions of PTS, but with a penalized root at variate dimension, i.e., $|sequence| \times \sqrt{|variate|}$ as we focus more on the sequence (temporal) dimension of PTS. For example, after an univariate time series $\mathbf{x}^{(i)} \in \mathbb{R}^{L\times 1}$ is patched into a PTS $\mathbf{x}_p^{(i)} \in \mathbb{R}^{N\times P}$, the resolution R_{PTS} is defined as:

$$R_{PTS} = N\sqrt{P} = (\lfloor \frac{L-P}{Str} \rfloor + 1)\sqrt{P} \approx \frac{\sqrt{P}}{Str} \tag{6}$$

In the last approximation, L is ignored as R_{PTS} aims to quantify the relative resolution.

4.2 Hybrid Mamba-Transformer Experts

Inspired by Mixture-of-Experts (MoEs) [11, 18, 33], we introduce a hybrid Mamba-Transformer experts architecture. Unlike ambiguous roles in traditional MoEs models, we assign clear patterns and variations roles to the two experts.

Patterns Expert. Mamba achieves impressive performance on tasks requiring scaling to long-range sequences as it introduces a selective mechanism to remember relevant patterns and filter out irrelevant noises indefinitely. Consequently, we incorporate the Mamba block as a expert to extract long-term patterns and filter out small variations in long-range time series. As shown in Figure 5, the patterns expert encodes long-range PTS $\mathbf{x}_{pL}^{(i)} \in \mathbb{R}^{N_L\times P_L}$ into high-dimension space $\mathbf{x}_L^{(i)} \in \mathbb{R}^{N_L\times D}$ in the encoding layer, and a Mamba block is followed. Mamba takes an input $\mathbf{x}_L^{(i)}$ and expands the dimension by two input linear projections. For one projection, Mamba processes the expanded embedding through a convolution and a SiLU activation before feeding into the SSM. The core discretized SSM module is able to select input-dependent patterns and filter out irrelevant variations. The other projection followed by SiLU activation, as a residual connection, is combined with output of the SSM module through a multiplicative gate. Finally, Mamba delivers output $\mathbf{z}_L^{(i)} \in \mathbb{R}^{N_L\times D}$ through an output linear projection. Note that we do not need a positional embedding typically existing in Transformer as Mamba's recurrence mechanism [16] inherently encodes positions.

Variations Expert. Transformer is potential to capture variations as discussed in the Section 3. However, it lacks locality inductive bias and quadratic complexity further impacts its adoption. Therefore, we propose a local window Transformer (LWT) with enhanced locality-awareness capabilities to capture short-term variations in short range. Figure 5 shows that the variations expert first projects short-range PTS $\mathbf{x}_{pS}^{(i)}$ and positional information into a embedding $\mathbf{x}_S^{(i)} \in \mathbb{R}^{N_S\times D}$. The embedding is fed into LWT where the local window attention forces each token only care surrounding tokens within the window. We denote the output of LWT as $\mathbf{z}_S^{(i)} \in \mathbb{R}^{N_S\times D}$. Figure 6 illustrates the mechanism of local window attention. Despite its strong local inductive bias, LWT maintains an extensive receptive field. By stacking multiple layers, the upper layers gain access to all input locations, enabling the construction of representations that integrate information across the entire input, similar to the capabilities of CNNs [44]. The LWT is able to increase the receptive field by stacking multiple layers, and the respective field

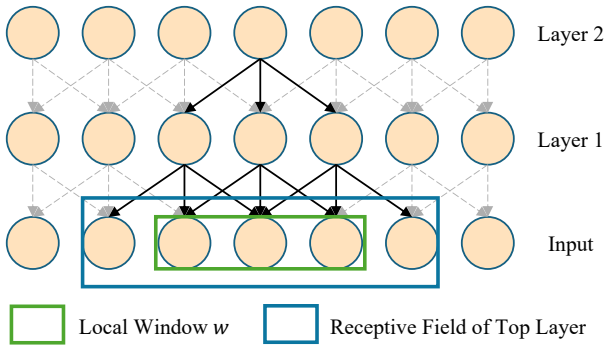


Figure 6: LWT employs a fixed-size window w to force each token only attend to local tokens within the window. By stacking multiple layers, the upper attention layer l can aggregate information from the lower layer and obtain a large receptive field $l \times w$. The complexity of Transformer can be reduced from quadratic to linear with the local window.

of layer l is $l \times w$. Moreover, the complexity reduces from $O(S^2)$ to $O(w * S)$ on the length of short-range time series (not considering the patching here).

4.3 Mixture of Experts

Inspired by the idea of MoE, we propose a long-short router to adaptively learn the contributions of the patterns expert and the variation expert.

Long-Short Router. In the MoE community, router [11] is a key module which primarily directs tokens to only a subset of experts for saving computational cost. Different from the path-assigning role of traditional MoE routers, the proposed long-short router is capable of learning the relative contributions of the two specialized experts, adaptively integrating long- and short-range time series representations. Formally, the router projects input time series $\mathcal{L} \in \mathbb{R}^{L \times M}$ into the D-dimension space, subsequently transformed by a flatten layer into a vector $\mathbf{z}_R$. A linear layer with a softmax function is followed to output two values $p_L, p_S \in (0, 1)$, indicating two weights of patterns and variations.

Forecasting Module. The forecasting module flattens long-range embedding $\mathbf{z}_L^{(i)}$ and short-range embedding $\mathbf{z}_S^{(i)}$ into a single-row vector and concatenates them with respective weights p_L and p_S. The resulting representation $\mathbf{z}_{LS}^i \in \mathbb{R}^{(N_S+N_L)LD}$ integrates long-term patterns and short-term variations. Finally, a linear head is employed to forecast $\hat{\mathbf{x}}^{(i)} = \{\hat{x}_{L+1}, \hat{x}_{L+2}, ..., \hat{x}_{L+F}\} \in \mathbb{R}^{F \times 1}$ for individual variate i.

4.4 Linear Complexity Analysis.

The complexity of SST comes form two parts: Mamba and LWT. Mamba which adopts RNN-like mechanism achieves the linear complexity $O(L)$ on long-range time series $\mathcal{L}$. Given the attention operation occupies $O(w^2)$ complexity within a local window w each time step, LWT consumes $O(w^2 * \frac{S}{w})$ complexity on short-range time series $\mathcal{S}$. Considering the use of multi-scale patcher, the total complexity of SST is $O(\frac{L}{N_L} + \frac{wS}{N_S})$. Note that w, N_S, and N_L are constant, and S is linear related to and smaller than L.

5 Experiments

In this section, we describe the details of experiments, including experimental setup, time series forecasting results, ablation studies, and analysis on memory and speed.

5.1 Experimental Setup

Datasets. To evaluate the proposed SST, we adopt seven popular real-world datasets [27], including ETTh1&ETTh2, ETTm1&ETTm2, Weather, ECL, and Traffic. The description of these multivariate time series datasets are summarized in Table 2.

Baselines and Metrics. We compare SST with representative time series forecasting methods within three years. They include Mamba-based models (TimeMachine [1], S-Mamba [42], and SiMBA [32]), Transformer-based forecasters (MTST [55], iTransformer [27], and PatchTST [30]), MLP-based framework (TimeMixer [41], RLinear [22], and DLinear [54]) . Note that it is unfair to include LLMs or large time series foundation models as they are pre-trained on a large corpus of data. To assess the performance of time series forecasters, we adopt two widely-used metrics MSE and MAE. The lower MSE and MAE indicate more accurate forecasting results.

Experimental Setting. We use low-resolution $R_{PTS} = 0.43$ ($P_L = 48$ and $Str_L = 16$) for long-range time series and high-resolution $R_{PTS} = 0.5$ ($P_L = 16$ and $Str_L = 8$) for short range. We set the look-back window length $L = 2S = 672$, and the future value length $F \in \{96, 192, 336, 720\}$. We run all experiments (including baselines) ten times with different seeds and report their average performance.

5.2 Time Series Forecasting Results.

Table 3 presents the main time series forecasting results. Overall, SST consistently outperforms baseline models, including Mamba-based, Transformer-based, and MLP-based approaches across all real-world datasets. For instance, on the ETTm1 dataset with the longest forecast horizon ($F = 720$), SST achieves an MSE of 0.429, compared to 0.488 for S-Mamba, 0.491 for iTransformer, and 0.487 for TimeMixer, reflecting relative improvements of 13.75%, 14.45%, and 13.51%, respectively. This superior performance highlights the effectiveness of SST's architectural design. By leveraging low-resolution, long-range representations, SST employs Mamba to extract global patterns, while using high-resolution, short-range representations allows the Transformer-based LWT module to capture local variations. A subsequent long-short router adaptively integrates long- and short-range dependencies, enabling more accurate forecasting. Notably, SST outperforms MTST [55] which employs Transformer to model diverse temporal patterns at different resolutions in multiple branches. This comparison suggests that modeling multi-scale resolution alone may not be the most effective method for time series forecasting.

5.3 Ablation Studies

To assess the contribution of each component in SST, we conduct ablation studies on the following variants: *Mamba*, *LWT*, *w/o Patcher* (SST without the patching module), and *w/o Router* (SST without the routing module), and also comparing SST with Mambaformer

Table 2: The statistics of seven real-world time series datasets.

Datasets	ETTh1	ETTh2	ETTm1	ETTm2	Weather	ECL	Traffic
Variates	7	7	7	7	21	321	862
Timestamps	17,420	17,420	69,680	69,680	52,696	26,304	17,544

Table 3: Multivariate time series forecasting results across seven datasets. The best and second-best performing models are highlighted in bold and underline.

Models		SST		TimeMachine		Mamba-based S-Mamba		SiMBA		Transformer-based MTST		iTransformer		PatchTST		TimeMixer		MLP-based RLinear		DLinear	
Metrics		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	**0.381**	0.405	0.389	0.402	0.392	**0.390**	0.432	0.416	**0.381**	0.392	0.386	0.405	0.414	0.419	0.403	0.421	0.386	0.395	0.386	0.400
	192	**0.430**	0.434	0.435	0.440	0.449	0.439	0.451	**0.422**	0.437	0.438	0.441	0.436	0.460	0.445	0.451	0.447	0.437	0.424	0.437	0.432
	336	0.443	**0.446**	0.450	0.448	0.467	0.481	0.477	0.465	**0.425**	0.466	0.487	0.458	0.501	0.466	0.517	0.451	0.479	**0.446**	0.481	0.459
	720	0.502	0.501	0.480	**0.465**	**0.475**	0.468	0.531	0.515	0.483	0.472	0.503	0.491	0.500	0.518	0.529	0.502	0.481	0.470	0.519	0.516
ETTh2	96	0.291	0.346	**0.230**	0.349	0.292	0.357	0.338	0.370	0.304	0.359	0.297	0.349	0.302	0.348	0.316	0.360	0.288	**0.338**	0.333	0.387
	192	**0.369**	0.397	0.371	0.400	0.380	0.402	0.410	0.465	0.381	0.395	0.380	0.400	0.388	0.400	0.402	0.415	0.374	**0.390**	0.477	0.476
	336	**0.374**	**0.414**	0.402	0.449	0.391	0.420	0.481	0.441	0.399	0.417	0.428	0.432	0.426	0.433	0.417	0.433	0.415	0.426	0.594	0.541
	720	**0.419**	0.447	0.425	0.438	0.437	0.455	0.486	0.501	0.426	**0.401**	0.427	0.445	0.431	0.446	0.445	0.460	0.420	0.440	0.831	0.657
ETTm1	96	**0.298**	**0.355**	0.312	0.371	0.311	0.380	0.357	0.381	0.317	0.358	0.334	0.368	0.329	0.367	0.349	0.381	0.355	0.376	0.345	0.372
	192	**0.347**	**0.381**	0.365	0.409	0.389	0.419	0.451	0.466	0.361	0.387	0.377	0.391	0.367	0.385	0.385	0.390	0.391	0.392	0.380	0.389
	336	**0.374**	0.397	0.421	0.410	0.401	0.417	0.483	0.511	0.383	**0.378**	0.426	0.420	0.399	0.410	0.415	0.402	0.424	0.415	0.413	0.413
	720	**0.429**	**0.428**	0.496	0.437	0.488	0.476	0.485	0.507	0.437	0.439	0.491	0.459	0.454	0.439	0.487	0.467	0.487	0.450	0.474	0.453
ETTm2	96	0.176	0.264	0.185	0.290	0.191	0.301	0.251	0.332	0.199	0.288	0.180	0.264	**0.175**	**0.259**	0.205	0.281	0.182	0.265	0.193	0.292
	192	**0.231**	0.303	0.292	0.309	0.253	0.312	0.287	0.312	0.257	0.328	0.250	0.309	0.241	**0.302**	0.257	0.312	0.246	0.304	0.284	0.362
	336	**0.290**	0.339	0.321	0.367	0.298	0.342	0.303	**0.337**	0.309	0.363	0.311	0.348	0.305	0.343	0.310	0.388	0.307	0.342	0.369	0.427
	720	**0.388**	**0.398**	0.401	0.400	0.409	0.407	0.420	0.431	0.395	0.416	0.412	0.407	0.402	0.400	0.401	0.414	0.407	**0.398**	0.554	0.522
Weather	96	**0.153**	0.205	0.174	0.218	0.169	0.221	0.187	**0.192**	0.181	0.239	0.174	0.214	0.177	0.218	0.190	0.239	0.192	0.232	0.196	0.255
	192	**0.196**	0.244	0.200	0.258	0.205	0.248	0.212	0.265	0.238	0.272	0.221	0.254	0.225	0.259	0.212	0.270	**0.240**	0.271	0.237	0.296
	336	**0.246**	**0.283**	0.280	0.299	0.288	0.299	0.272	0.288	0.285	0.315	0.278	0.296	0.278	0.297	0.271	0.308	0.292	0.307	0.283	0.335
	720	**0.314**	**0.334**	0.352	0.359	0.335	0.369	0.360	0.369	0.352	0.377	0.358	0.347	0.354	0.348	0.358	0.363	0.364	0.353	0.345	0.381
ECL	96	**0.141**	**0.239**	0.156	0.240	0.157	0.255	0.167	0.261	0.171	0.264	0.148	0.240	0.181	0.270	0.181	0.275	0.201	0.281	0.197	0.282
	192	**0.159**	0.255	0.161	0.268	0.188	0.271	0.187	**0.253**	0.188	0.281	0.162	**0.253**	0.188	0.274	0.160	0.254	0.201	0.283	0.196	0.285
	336	**0.171**	**0.268**	0.195	0.272	0.192	0.275	0.199	0.284	0.207	0.298	0.178	0.269	0.204	0.293	0.191	0.286	0.215	0.298	0.209	0.301
	720	**0.208**	**0.300**	0.231	0.307	0.241	0.339	0.250	0.314	0.244	0.333	0.225	0.317	0.246	0.324	0.247	0.318	0.257	0.331	0.245	0.333
Traffic	96	**0.367**	**0.257**	0.395	0.268	0.401	0.259	0.511	0.342	0.392	0.276	0.395	0.268	0.462	0.295	0.412	0.288	0.649	0.389	0.650	0.396
	192	0.385	**0.266**	0.393	0.282	0.389	0.294	0.492	0.377	0.419	0.297	0.417	0.276	**0.366**	0.296	0.468	0.304	0.601	0.366	0.598	0.370
	336	**0.401**	**0.275**	0.443	0.368	0.427	0.296	0.530	0.351	0.434	0.304	0.433	0.283	0.482	0.304	0.436	0.304	0.609	0.369	0.605	0.373
	720	**0.445**	**0.302**	0.470	0.309	0.473	0.347	0.547	0.367	0.471	0.324	0.467	**0.302**	0.514	0.322	0.474	0.307	0.647	0.387	0.645	0.394

family. Results are presented in Figure 7, and we draw the following observations: (1) SST significantly outperforms both *Mamba* and *LWT*, highlighting the advantage of combining Mamba and Transformer. This hybrid design enables SST to effectively capture long-range patterns and short-range variations, validating the benefit of integrating state space and attention mechanisms. (2) SST also surpasses *w/o Patcher* and *w/o Router*, demonstrating the importance of the multi-scale patcher and dynamic router. The patcher facilitates resolution-aware processing by adapting time series granularity for long- and short-range segments, while the router dynamically balances contributions from the two expert modules. These results confirm the effectiveness of SST's multi-scale design and MoE-based hybrid architecture. (3) Compared to the Mambaformer family, SST achieves significantly better performance, further emphasizing the benefits of its carefully integrated hybrid components.

5.4 Memory and Speed Analysis

To check the efficiency of SST in practice, we conduct memory and speed analysis. As shown in Figure 8, we depict figures where consumed memory and the elapsed time each epoch vary with the length of input time series L. We compare SST with PatchTST [30] and vanilla Transformer [39]. Note that we do not compare iTransformer because its attention operates on variate dimension instead of sequence dimension. All the computations were performed on 24 GB NVIDIA RTX A5000 GPU at Ubuntu 20.04.6 LTS. From the figure, we observe SST is efficient and has promising scalability on time steps. SST is able to scale linearly to 6k time steps. The impressive scalability stems from the fact that Mamba implement a hardware-aware algorithm, and LWT leverages a fixed-size sliding window to operate attentions. In contrast, vanilla Transformer struggles in quadratic complexity, significantly preventing it from attending to long time series. In the experiments, vanilla Transformer can only attend to maximum 336 time steps. It cannot continue increasing input length due to the OOM (Out-Of-Memory) issue. PatchTST uses patch techniques to reduce complexity by a factor of stride, thus alleviating the issue to a degree. However, it still fall short of scalability and can only scale to 3k time steps, much lower than 6k time steps of SST. The memory and speed analysis demonstrate efficiency of SST in practice.

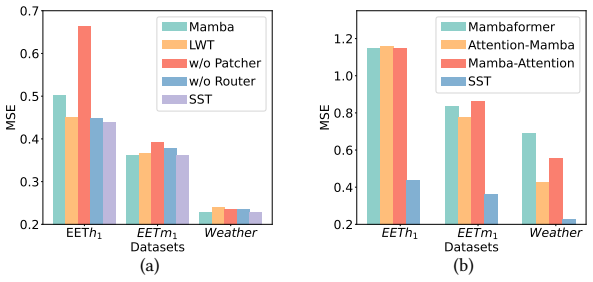


Figure 7: Ablation study on three datasets for (a) exclusion of key components; (b) comparison with Mambaformer family.

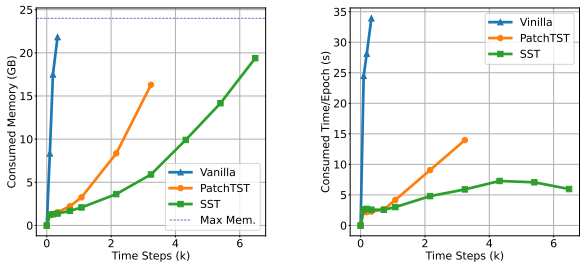


Figure 8: Comparison of consumed memory and the elapsed time of each epoch for vanilla Transformer, PatchTST, and SST. The OOM (out-of-memory) issues emerge for vanilla transformer, PatchTST, and SST when the length of input is 336, 3240, and 6480 time steps. It demonstrates the efficiency of SST in terms of both memory and time.

6 Related Work

This work is primarily related to two lines of research: (1) time series forecasting and (2) state space models.

6.1 Time Series Forecasting

Time series forecasting has long been a fundamental problem. Classical deep learning approaches such as RNNs [6, 17] and LSTMs [47, 52] have been widely applied, but their effectiveness on long-range sequences is limited due to gradient vanishing issues [36]. Inspired by the success of Transformers [39] in natural language processing, numerous Transformer-based models have been adapted for time series forecasting [21, 26, 30, 43, 45, 56, 57]. For example, iTransformer [27] achieves state-of-the-art performance by applying attention and feed-forward networks on the inverted dimensions. More recently, the powerful large language models (LLMs) have been explored for time series forecasting [12, 19, 34]. However, these approaches remain computationally expensive due to their super-linear complexity and a huge amount of parameters contained in the model. In parallel, the ideas of multi-scale time series has been increasingly adopted. MTST [55] and TimeMixer(++) extract multi-scale features for predicative analysis. Although they also consider different time series granularities, they do not design different models for specific granularities. Different from them, our work employs different specifically designed models to capture patterns in long range and fluctuations in short range.

6.2 State Space Models

SSMs [13, 15, 16] have recently emerged as a powerful family of sequence modeling architectures. Mamba, a selective SSM equipped with a hardware-efficient algorithm, has demonstrated impressive performance across a wide range of modalities, including language [9, 13], vision [35, 53, 58], medicine [28, 46], graphs [2, 40], recommendation systems [25, 51], and time series [1, 23, 32, 42]. Recent efforts have proposed leveraging SSMs for time series forecasting [1, 32, 42]. For instance, TimeMachine [1] uses stacked Mamba blocks to model long-range dependencies in multivariate time series. However, these models are limited in capturing fine-grained variations because of the inherent lossy memory mechanism. In contrast, our work makes the first attempt to integrate Mamba and Transformer within a unified architecture for time series forecasting, aiming to capture both long- and short-range temporal dynamics, and also to eliminate *information interference* issue in the straightforward stacking of Mamba and Transformer.

A notable research direction involves hybridizing Mamba with Transformers. For example, Mambaformer [31] demonstrates strong performance in in-context learning tasks by interleaving Mamba and attention layers. Jamba [24] is a production-scale hybrid model that combines attention and SSMs, supporting up to 52B parameters and long-context modeling. While these hybrid approaches have focused on language modeling, our work is the first to explore a hybrid Mamba–Transformer architecture for time series forecasting, leveraging their complementary strengths to handle both long-term dependencies and short-term variations.

7 Conclusion and Future Work

In this paper, we analyze the strengths and weaknesses of Mamba and Transformer architectures: Mamba offers linear complexity and computational efficiency but may struggle with information retention, while Transformer provides direct access to past information at the cost of quadratic complexity. Motivated by this, we raise a research question: *Can we design a hybrid Mamba-Transformer architecture that is both effective and efficient for time series forecasting?* To explore this, we adapt Mambaformer to time series forecasting, which is the hybrid Mamba-Transformer architecture by stacking Mamba and attention layers for language modeling. The results show that such integration is ineffective for time series due to the *information interference* issue. To address the issue, our proposed SST leverages Mamba to extract long-term patterns in coarse-grained time series and Transformer to capture short-term variations in fine-grained time series. Extensive experiments demonstrate that our approach achieves state-of-the-art (SOTA) performance while maintaining linear complexity $O(L)$. Future work explores the application of hybrid Mamba-Transformer architectures to other time series analysis tasks such as classification and anomaly detection.

Acknowledgments

This material is based upon work supported by NSF awards (SaTC-2241068, IIS-2506643, and POSE-2346158), a Cisco Research Award, and a Microsoft Accelerate Foundation Models Research Award.

8 GenAI Usage Disclosure

This paper presents original work. After completing the manuscript, we used the GenAI tool ChatGPT-4o to assist with polishing the abstract and introduction.

References

[1] Md Atik Ahamed and Qiang Cheng. 2024. Timemachine: A time series is worth 4 mambas for long-term forecasting. *arXiv preprint arXiv:2403.09898* (2024).

[2] Ali Behrouz and Farnoosh Hashemi. 2024. Graph Mamba: Towards Learning on Graphs with State Space Models. *arXiv preprint arXiv:2402.08678* (2024).

[3] Ronald Boellaard, Nanda C Krak, Otto S Hoekstra, and Adriaan A Lammertsma. 2004. Effects of noise, image resolution, and ROI definition on the accuracy of standard uptake values: a simulation study. *Journal of Nuclear Medicine* 45, 9 (2004), 1519–1527.

[4] Ching Chang, Wen-Chih Peng, and Tien-Fu Chen. 2023. Llm4ts: Two-stage fine-tuning for time-series forecasting with pre-trained llms. *arXiv preprint arXiv:2308.08469* (2023).

[5] Zihan Chen, Lei Nico Zheng, Cheng Lu, Jialu Yuan, and Di Zhu. 2023. ChatGPT informed graph neural network for stock movement prediction. *arXiv preprint arXiv:2306.03763* (2023).

[6] Xingyi Cheng, Ruiqing Zhang, Jie Zhou, and Wei Xu. 2018. Deeptransport: Learning spatial-temporal dependency for traffic condition forecasting. In *2018 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.

[7] Robert B Cleveland, William S Cleveland, Jean E McRae, Irma Terpenning, et al. 1990. STL: A seasonal-trend decomposition. *J. off. Stat* 6, 1 (1990), 3–73.

[8] Elkin Cruz-Camacho, Kevin A Brown, Xin Wang, Xiongxiao Xu, Kai Shu, Zhiling Lan, Robert B Ross, and Christopher D Carothers. 2023. Hybrid PDES Simulation of HPC Networks Using Zombie Packets. In *Proceedings of the 2023 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*. 128–132.

[9] Tri Dao and Albert Gu. 2024. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060* (2024).

[10] Mahan Fathi, Jonathan Pilault, Pierre-Luc Bacon, Christopher Pal, Orhan Firat, and Ross Goroshin. 2023. Block-state transformer. *arXiv preprint arXiv:2306.09539* (2023).

[11] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[12] Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew G Wilson. 2024. Large language models are zero-shot time series forecasters. *Advances in Neural Information Processing Systems* 36 (2024).

[13] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).

[14] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. 2020. Hippo: Recurrent memory with optimal polynomial projections. *Advances in neural information processing systems* 33 (2020), 1474–1487.

[15] Albert Gu, Karan Goel, and Christopher Re. 2022. Efficiently Modeling Long Sequences with Structured State Spaces. In *International Conference on Learning Representations*. https://openreview.net/forum?id=uYLFoz1vlAC

[16] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. 2021. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in neural information processing systems* 34 (2021), 572–585.

[17] Hansika Hewamalage, Christoph Bergmeir, and Kasun Bandara. 2021. Recurrent neural networks for time series forecasting: Current status and future directions. *International Journal of Forecasting* 37, 1 (2021), 388–427.

[18] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. 1991. Adaptive mixtures of local experts. *Neural computation* 3, 1 (1991), 79–87.

[19] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. 2023. Time-llm: Time series forecasting by reprogramming large language models. *arXiv preprint arXiv:2310.01728* (2023).

[20] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. 2021. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International conference on learning representations*.

[21] Shiyang Li, Xiaoyong Jin, Yao Xuan, Xiyou Zhou, Wenhu Chen, Yu-Xiang Wang, and Xifeng Yan. 2019. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. *Advances in neural information processing systems* 32 (2019).

[22] Zhe Li, Shiyi Qi, Yiduo Li, and Zenglin Xu. 2023. Revisiting long-term time series forecasting: An investigation on linear mapping. *arXiv preprint arXiv:2305.10721* (2023).

[23] Aobo Liang, Xingguo Jiang, Yan Sun, and Chang Lu. 2024. Bi-Mamba4TS: Bidirectional Mamba for Time Series Forecasting. *arXiv preprint arXiv:2404.15772* (2024).

[24] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. 2024. Jamba: A Hybrid Transformer-Mamba Language Model. arXiv:2403.19887 [cs.CL]

[25] Chengkai Liu, Jianghao Lin, Jianling Wang, Hanzhou Liu, and James Caverlee. 2024. Mamba4Rec: Towards Efficient Sequential Recommendation with Selective State Space Models. *arXiv preprint arXiv:2403.03900* (2024).

[26] Shizhan Liu, Hang Yu, Cong Liao, Jianguo Li, Weiyao Lin, Alex X Liu, and Schahram Dustdar. 2021. Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting. In *International conference on learning representations*.

[27] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=JePfAI8fah

[28] Jun Ma, Feifei Li, and Bo Wang. 2024. U-mamba: Enhancing long-range dependency for biomedical image segmentation. *arXiv preprint arXiv:2401.04722* (2024).

[29] Larry R Medsker, Lakhmi Jain, et al. 2001. Recurrent neural networks. *Design and Applications* 5, 64-67 (2001), 2.

[30] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=Jbdc0vTOcol

[31] Jongho Park, Jaeseung Park, Zheyang Xiong, Nayoung Lee, Jaewoong Cho, Samet Oymak, Kangwook Lee, and Dimitris Papailiopoulos. 2024. Can Mamba Learn How to Learn? A Comparative Study on In-Context Learning Tasks. *arXiv preprint arXiv:2402.04248* (2024).

[32] Badri N Patro and Vijay S Agneeswaran. 2024. SiMBA: Simplified Mamba-Based Architecture for Vision and Multivariate Time series. *arXiv preprint arXiv:2403.15360* (2024).

[33] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* (2017).

[34] Mingtian Tan, Mike A Merrill, Vinayak Gupta, Tim Althoff, and Thomas Hartvigsen. 2024. Are Language Models Actually Useful for Time Series Forecasting? *arXiv preprint arXiv:2406.16964* (2024).

[35] Yujin Tang, Peijie Dong, Zhenheng Tang, Xiaowen Chu, and Junwei Liang. 2024. VMRNN: Integrating Vision Mamba and LSTM for Efficient and Accurate Spatiotemporal Forecasting. arXiv:2403.16536 [cs.CV]

[36] Igor V Tetko, David J Livingstone, and Alexander I Luik. 1995. Neural network studies. 1. Comparison of overfitting and overtraining. *Journal of chemical information and computer sciences* 35, 5 (1995), 826–833.

[37] Marina Theodosiou. 2011. Forecasting monthly and quarterly time series using STL decomposition. *International Journal of Forecasting* 27, 4 (2011), 1178–1195.

[38] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. 2016. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022* (2016).

[39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[40] Chloe Wang, Oleksii Tsepa, Jun Ma, and Bo Wang. 2024. Graph-mamba: Towards long-range graph sequence modeling with selective state spaces. *arXiv preprint arXiv:2402.00789* (2024).

[41] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y Zhang, and Jun Zhou. 2024. Timemixer: Decomposable multiscale mixing for time series forecasting. *arXiv preprint arXiv:2405.14616* (2024).

[42] Zihan Wang, Fanheng Kong, Shi Feng, Ming Wang, Han Zhao, Daling Wang, and Yifei Zhang. 2024. Is Mamba Effective for Time Series Forecasting? *arXiv preprint arXiv:2403.11144* (2024).

[43] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. 2023. Transformers in Time Series: A Survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*, Edith Elkind (Ed.). International Joint Conferences on Artificial Intelligence Organization, 6778–6786. doi:10.24963/ijcai.2023/759 Survey Track.

[44] Felix Wu, Angela Fan, Alexei Baevski, Yann N Dauphin, and Michael Auli. 2019. Pay less attention with lightweight and dynamic convolutions. *arXiv preprint arXiv:1901.10430* (2019).

[45] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems* 34 (2021), 22419–22430.

[46] Zhaohu Xing, Tian Ye, Yijun Yang, Guang Liu, and Lei Zhu. 2024. Segmamba: Long-range sequential modeling mamba for 3d medical image segmentation. *arXiv preprint arXiv:2401.13560* (2024).

[47] Xiongxiao Xu, Kevin A. Brown, Tanwi Mallick, Xin Wang, Elkin Cruz-Camacho, Robert B. Ross, Christopher D. Carothers, Zhiling Lan, and Kai Shu. 2024. Surrogate Modeling for HPC Application Iteration Times Forecasting with Network Features. In *Proceedings of the 38th ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*. 93–97.

[48] Xiongxiao Xu, Solomon Abera Bekele, Brice Videau, and Kai Shu. 2024. Online energy optimization in gpus: A multi-armed bandit approach. *arXiv preprint arXiv:2410.11855* (2024).

[49] Xiongxiao Xu, Xin Wang, Elkin Cruz-Camacho, Christopher D. Carothers, Kevin A. Brown, Robert B. Ross, Zhiling Lan, and Kai Shu. 2023. Machine Learning for Interconnect Network Traffic Forecasting: Investigation and Exploitation. In *Proceedings of the 2023 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*. 133–137.

[50] Xiongxiao Xu, Yue Zhao, S Yu Philip, and Kai Shu. 2025. Beyond Numbers: A Survey of Time Series Analysis in the Era of Multimodal LLMs. *Authorea Preprints* (2025).

[51] Jiyuan Yang, Yuanzi Li, Jingyu Zhao, Hanbing Wang, Muyang Ma, Jun Ma, Zhaochun Ren, Mengqi Zhang, Xin Xin, Zhumin Chen, et al. 2024. Uncovering Selective State Space Model's Capabilities in Lifelong Sequential Recommendation. *arXiv preprint arXiv:2403.16371* (2024).

[52] Huaxiu Yao, Xianfeng Tang, Hua Wei, Guanjie Zheng, and Zhenhui Li. 2019. Revisiting spatial-temporal similarity: A deep learning framework for traffic prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 5668–5675.

[53] Weihao Yu and Xinchao Wang. 2024. MambaOut: Do We Really Need Mamba for Vision? *arXiv preprint arXiv:2405.07992* (2024).

[54] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 11121–11128.

[55] Yitian Zhang, Liheng Ma, Soumyasundar Pal, Yingxue Zhang, and Mark Coates. 2024. Multi-resolution time-series transformer for long-term forecasting. In *International conference on artificial intelligence and statistics*. PMLR, 4222–4230.

[56] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 11106–11115.

[57] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International Conference on Machine Learning*. PMLR, 27268–27286.

[58] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. 2024. Vision mamba: Efficient visual representation learning with bidirectional state space model. *arXiv preprint arXiv:2401.09417* (2024).

[59] Xun Zhu, Yutong Xiong, Ming Wu, Gaozhen Nie, Bin Zhang, and Ziheng Yang. 2023. Weather2k: A multivariate spatio-temporal benchmark dataset for meteorological forecasting based on real-time observation data from ground weather stations. *arXiv preprint arXiv:2302.10493* (2023).